

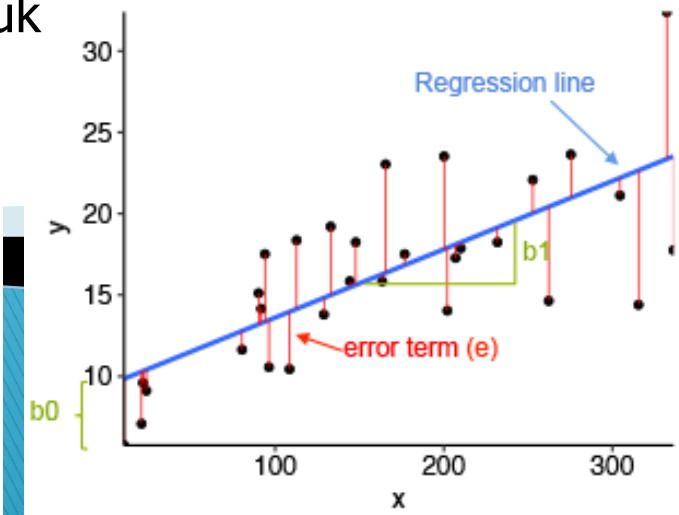
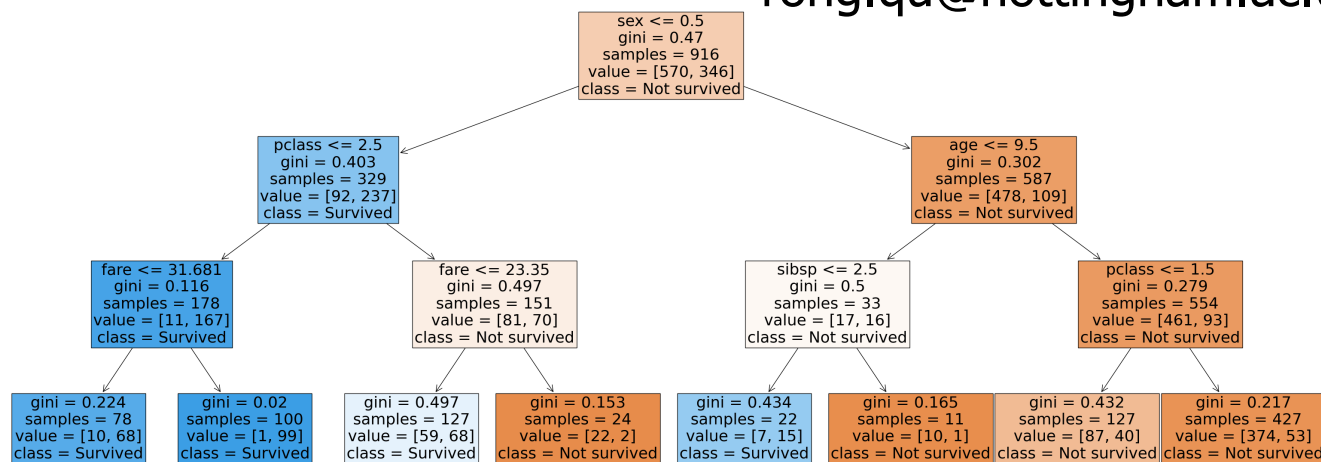
COMP1008 Part II: Machine Learning

- **Machine learning I:** Intro
 - **Machine learning II:** Process & Data, Un-supervised learning
 - **Machine learning III:** Supervised learning (this week)
 - **Machine learning IV:** ANN & extensions
 - Computing session 2: Data processing & visualization
- 

COMP1008 - Fundamentals of AI

Machine Learning: Supervised Learning

Professor Rong Qu
rong.qu@nottingham.ac.uk



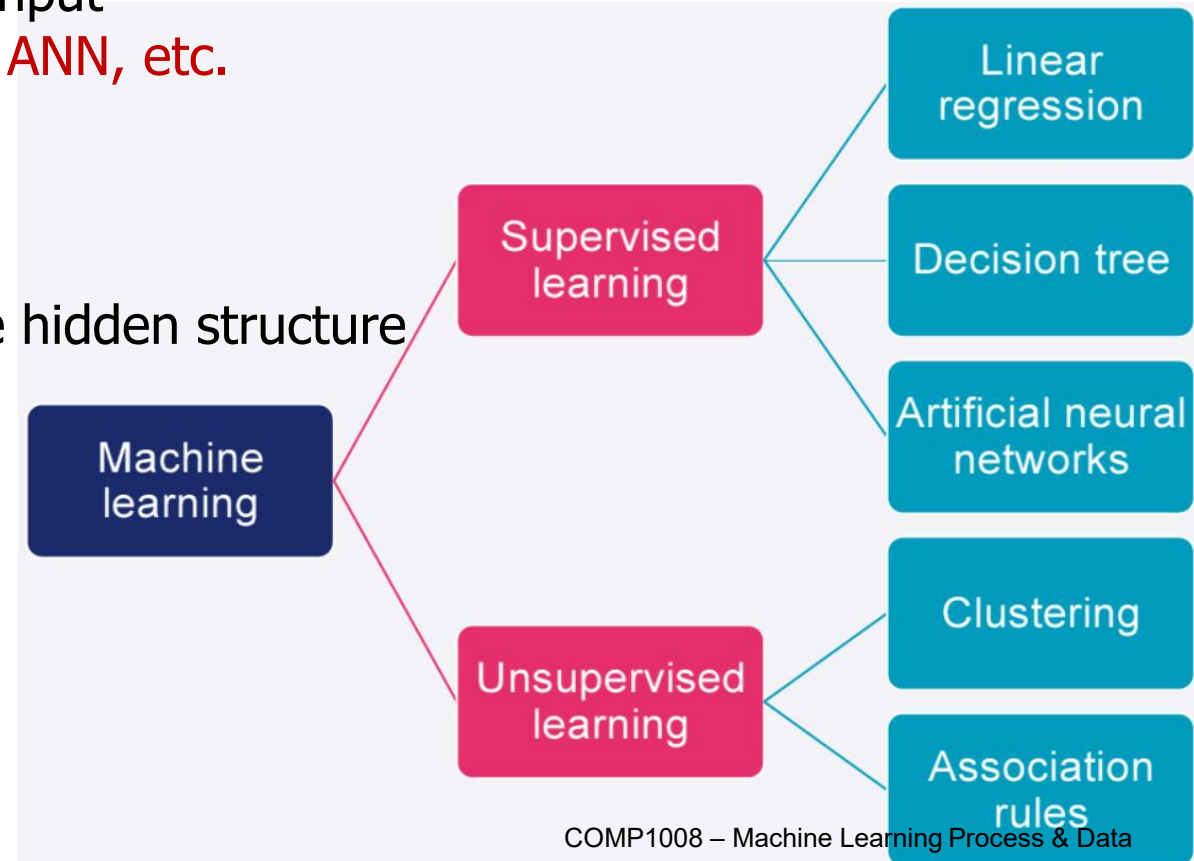
Machine Learning : (Main) Types of Tasks

▶ Supervised learning

- Task: learns from **labelled** training data (each sample/instance/data point consists of an input and a **desired** output)
- Goal: find/train a model to match output to input
- Algorithms: **Linear regression, Decision tree, ANN, etc.**
- Examples: classifications, predictions

▶ Unsupervised learning

- Task: learns from **unlabelled** data to describe hidden structure
- Goal: task dependent
- Algorithms: **K-means**
- Examples: Clustering/grouping
Association rules



Process: supervised learning

► Data preparation

prepare the input and output datasets

$y = df['Risk']$ # the class labels

$X = df.drop('Risk', axis=1)$ # the input features

ID	Age	Sex	Job	Housing	SavingAcc	CheckAcco	CreditAm	Duration	Purpose	Risk
0	67	male	2	own	NA	little	1169	6	radio/TV	good
1	22	female	2	own	little	moderate	5951	48	radio/TV	bad
2	49	male	1	own	little	NA	2096	12	education	good
3	45	male	2	free	little	little	7882	42	furniture/eq	good
4	53	male	2	free	little	little	4870	24	car	bad
5	35	male	1	free	NA	NA	9055	36	education	good

Samples/
Observations/
Instances

Features/Attributes

Labels/Targets

1. Data preparation

Data Pre-
processing



Process: supervised learning

1. Partition the total dataset into training & testing subsets

- 70% training vs. 30% testing

Computing sessions 2&3



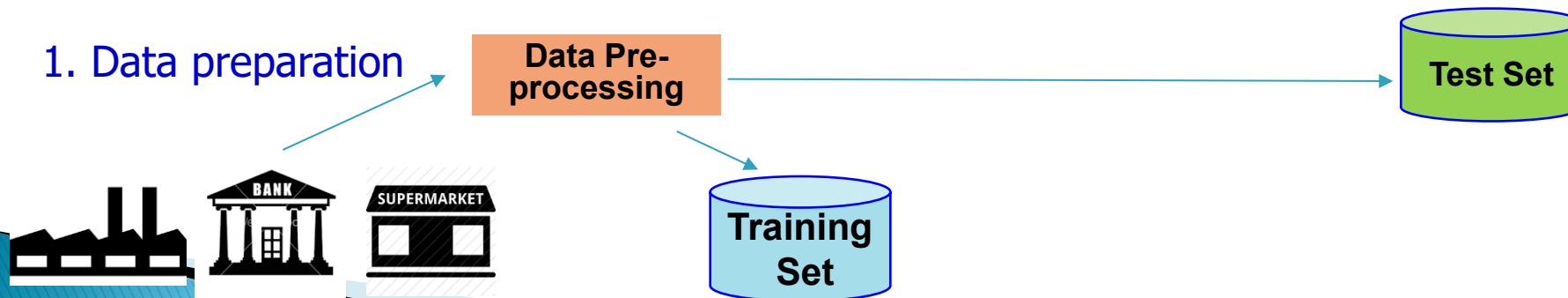
```
# import scikit-learn "model_selection"
```

```
from sklearn.model_selection import train_test_split
```

```
# prepare the training and testing datasets
```

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.3)
```

1. Data preparation



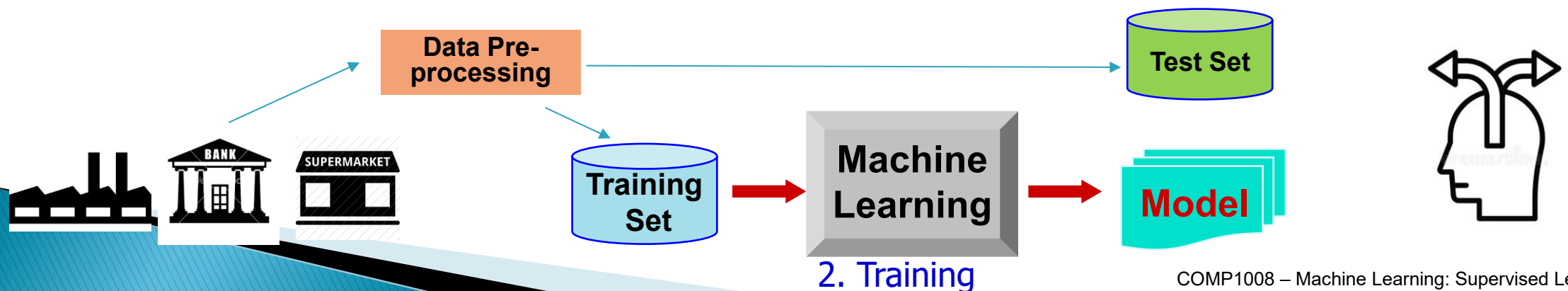
Process: supervised learning

2. Training the model on the training set

```
# import relevant scikit-learn module, e.g. linear_model, DecisionTreeClassifier  
from sklearn.xxx import yyy_ML_Algorithm
```

```
# instantiate the machine learning model  
ml_model1 = yyy_ML_Algorithm()
```

```
# train the model with the training data  
ml_model1.fit(x_train, y_train)
```

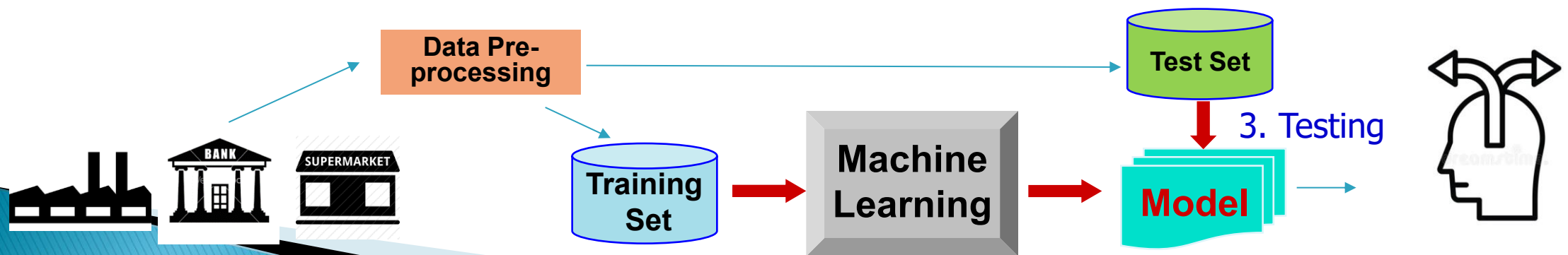


Process: supervised learning

3. Testing: evaluation of models on the test set

- How accurate the trained model works on the test data

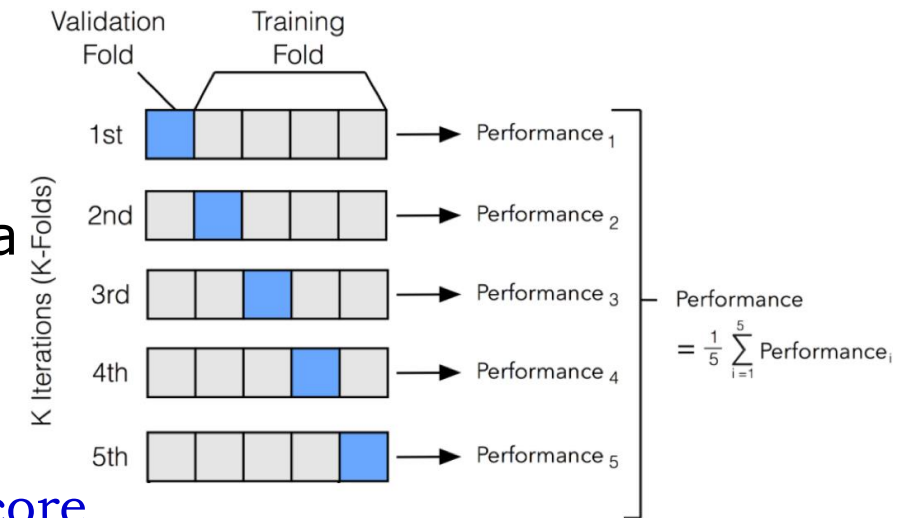
```
# import scikit-learn "metrics"
from sklearn import metrics
# predict on the testing data (input)
y_pred = ml_model1.predict(x_test)
# Calculate the model performance using MSE
print(metrics.mean_squared_error(y_test, y_pred))
```



Process: supervised learning

3. Testing: evaluation of models on the test set

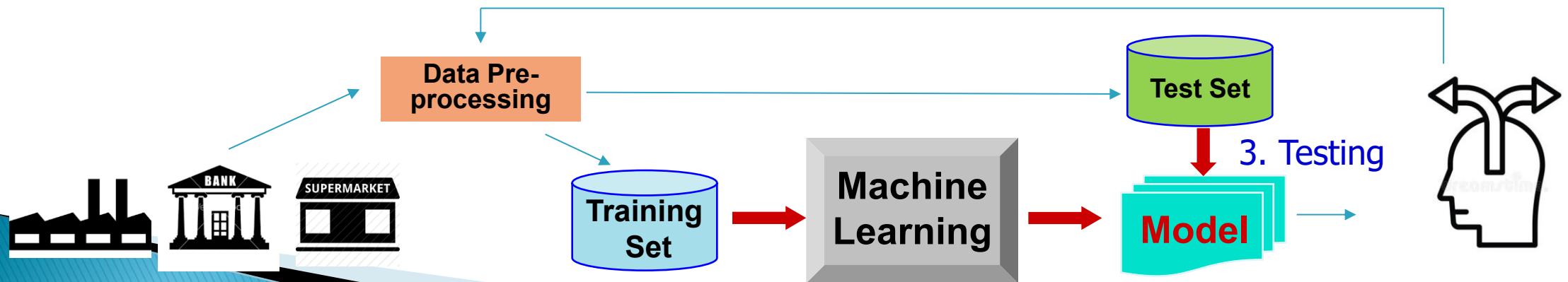
- How accurate the trained model works on the test data
- Multiple runs using different partitions, i.e. **K-fold cross-validation**



```
from sklearn.model_selection import cross_val_score
```

```
# k-fold cross-validation on the whole dataset
```

```
cross_val_score(ml_model2, X, y, cv=5,  
                  scoring='neg_mean_squared_error').mean()*-1
```



Regression: model

Supervised

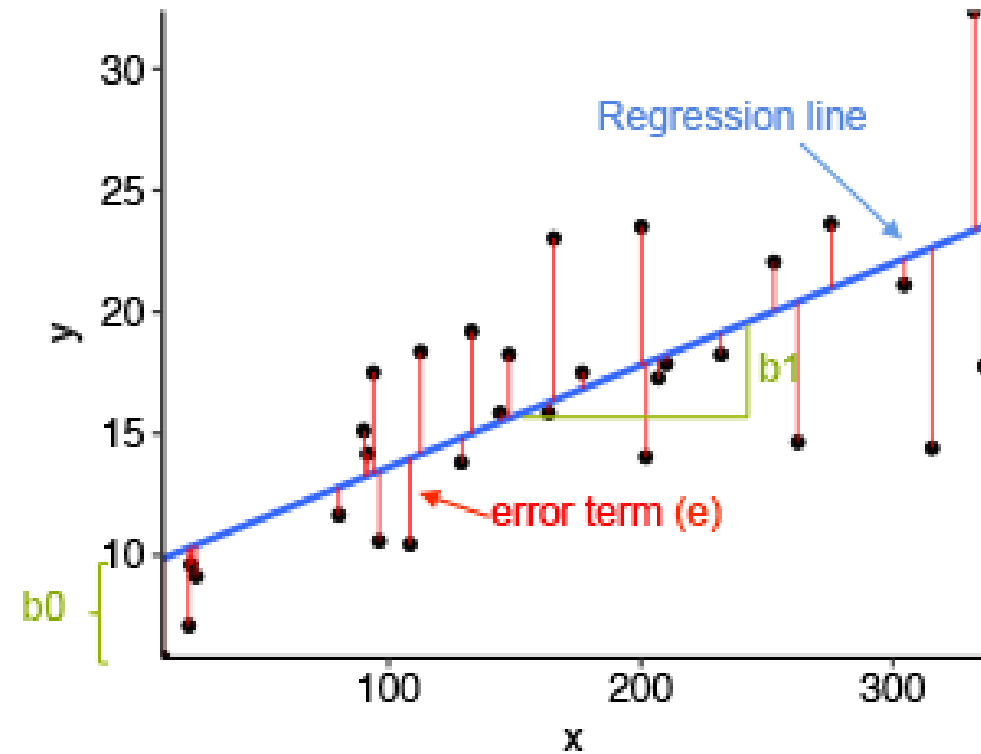
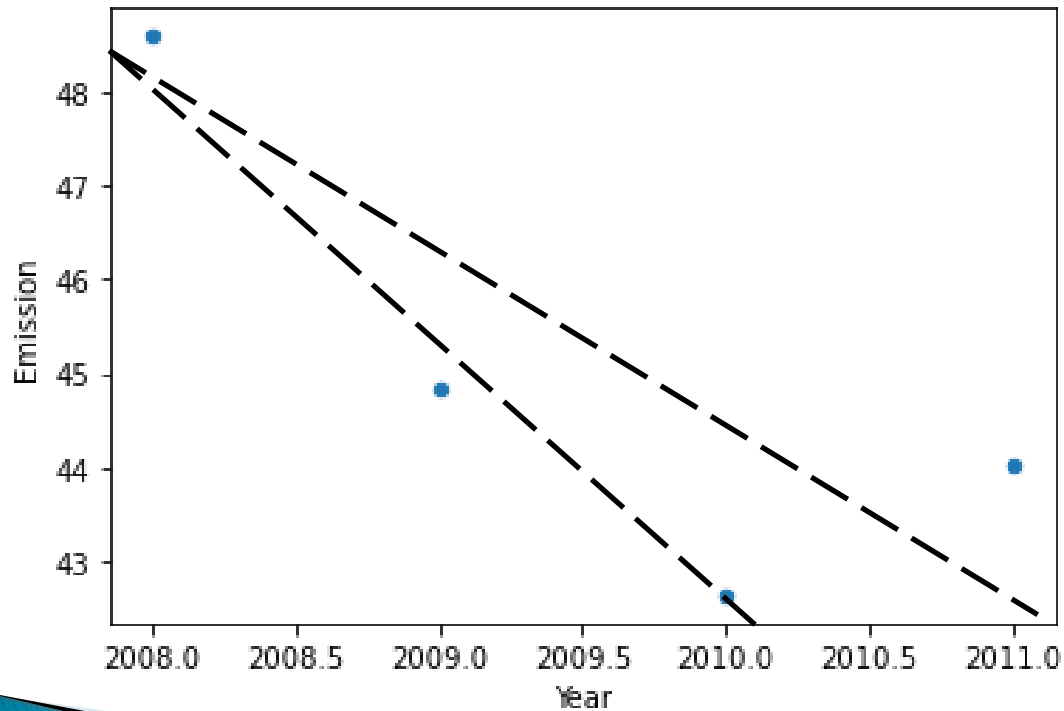
$y = F(x)$: relationship

D : labeled training set

Learn: $G(x)$: model to predict labels of new data

Goal: $[(F(x) - G(x))^2] \approx 0$

- Relationship between variable Y and variable(s) X
- Regression** training: based on the given data find the function (b_0 and b_1) that minimizes its mean squared error (MSE) to 'fit' the samples
 - Regression function: $G(x) = b_0 + b_1 * x$



Regression: data

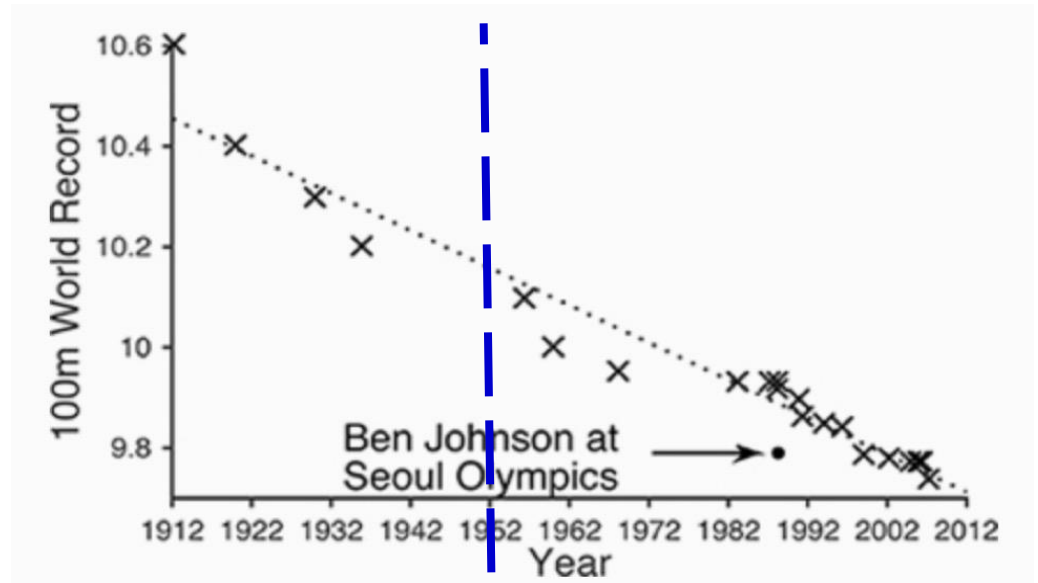
- Python library `sklearn`

```
# import the scikit-learn module "model_selection"  
from sklearn.model_selection import train_test_split
```

```
# prepare the training and testing datasets
```

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.3)
```

Computing session 3



Features	Labels
x_train ...	y_train
x_test ...	y_test

Used to learn **model parameters**



How well does the model **generalize** on unseen test instances?

Regression: training

```
# import the scikit-learn module "linear_model"
from sklearn.linear_model import LinearRegression
```

```
# instantiate the linear regression algorithm
lr = LinearRegression()
```

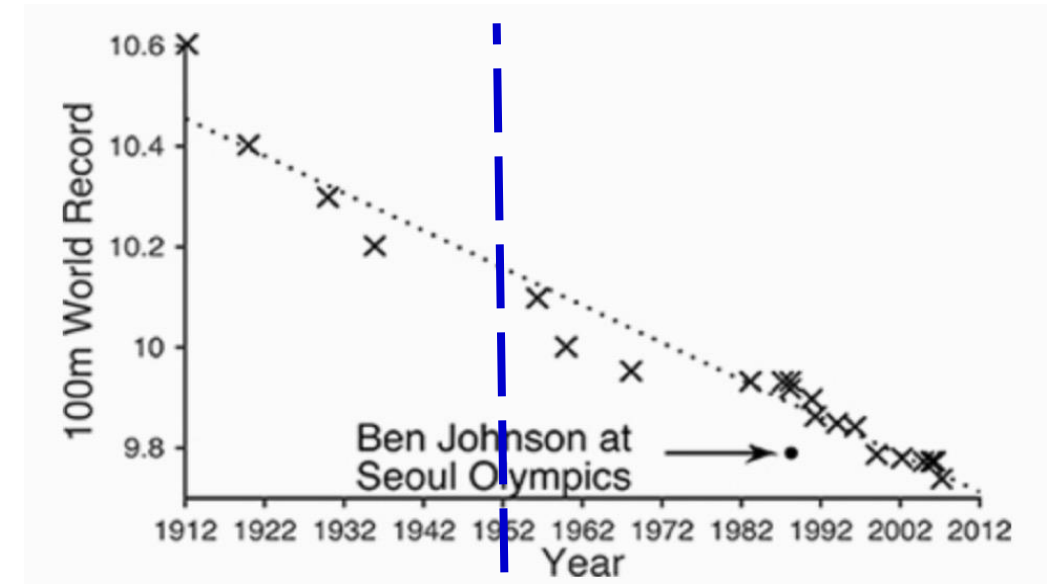
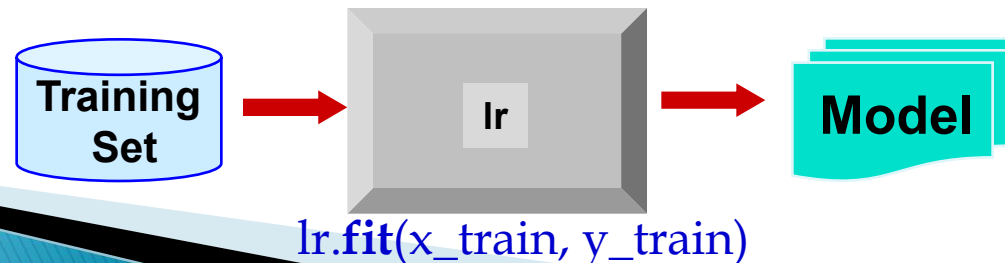
```
# train the algorithm with the training data prepared
lr.fit(x_train, y_train)
```

```
print('Coefficient/slope:', lr.coef_)
```

```
print('Intercept:', lr.intercept_)
```

```
print('The linear regression function learned between x and y is:
      y =', lr.intercept_, '+', lr.coef_[0], '* x')
```

Used to learn **model parameters**



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values



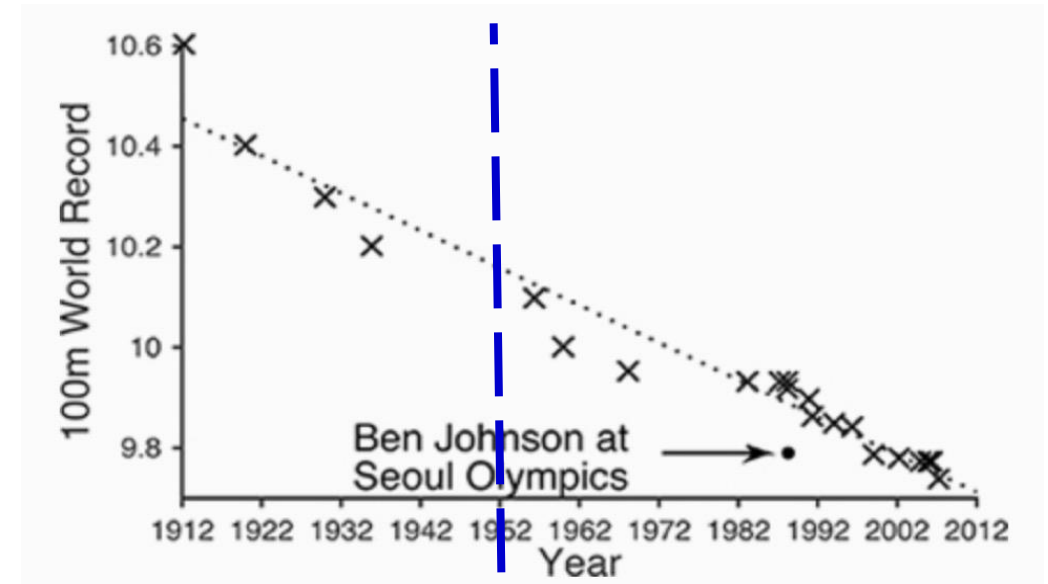
How well does the model **generalize** on unseen test instances?

Regression: testing

```
# scikit-learn "metrics" evaluate the trained model  
from sklearn import metrics
```

```
# predict with the model on testing data (input)  
y_pred = lr.predict(x_test)
```

```
# calculate model performance MSE, predicted  
# output vs. the actual output of test data prepared  
print(metrics.mean_squared_error(y_test, y_pred))
```



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

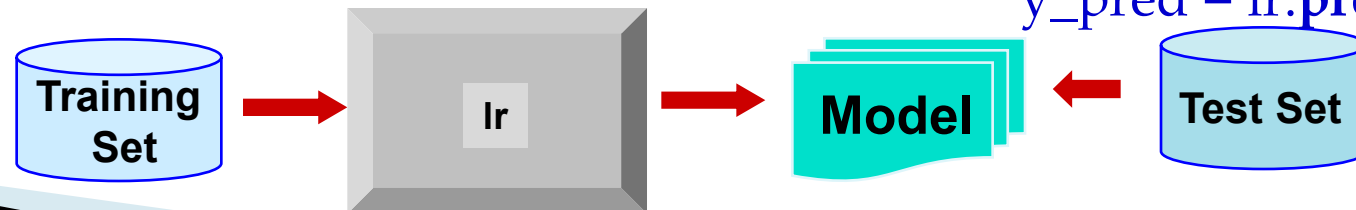
n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

Features	Labels
x_train ...	y_train
x_test ...	y_test

Used to learn **model parameters**



How well does the model **generalize** on unseen test instances?

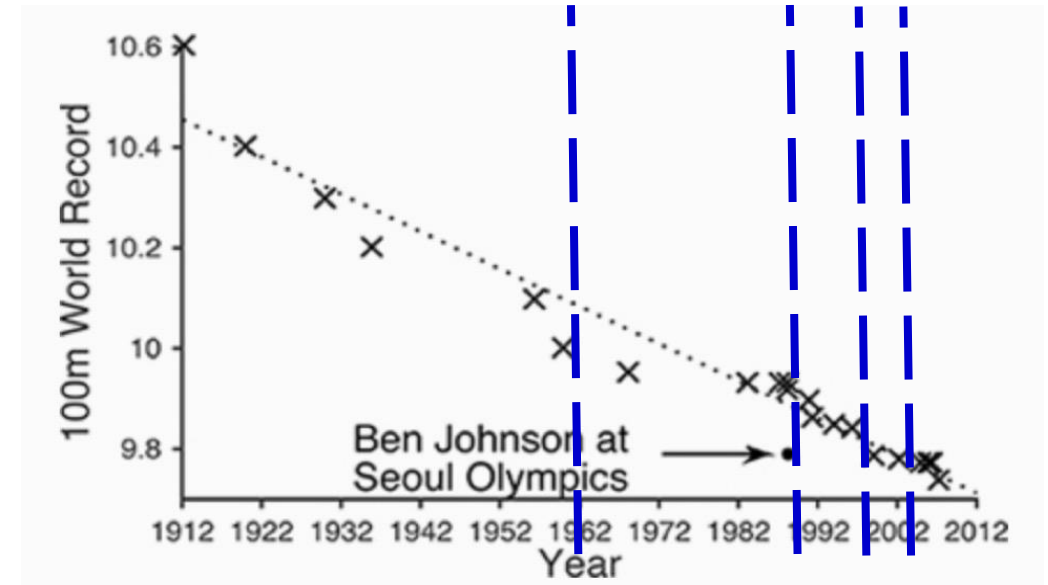
Regression: testing

```
# import the scikit-learn module "cross_val_score"
from sklearn.model_selection import cross_val_score
```

```
lr1 = LinearRegression()
```

```
# conduct k-fold cross-validation on the whole dataset
```

```
cross_val_score(lr1, X, y, cv=5, scoring='neg_mean_squared_error').mean()*-1
```



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Features	Labels
X	y

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

Used to learn **model parameters**



How well does the model **generalize** on unseen test instances?

Regression: testing

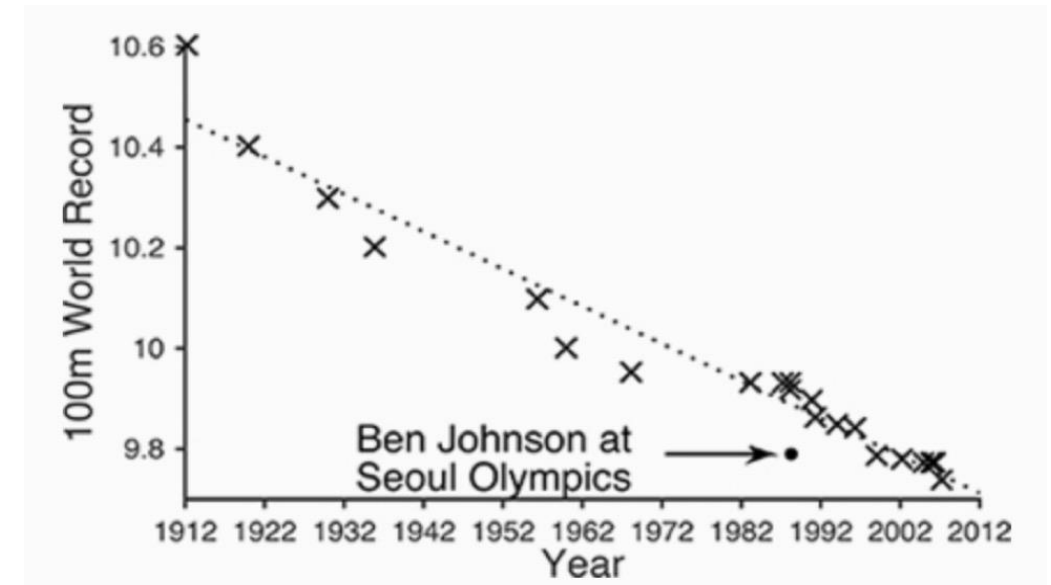
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

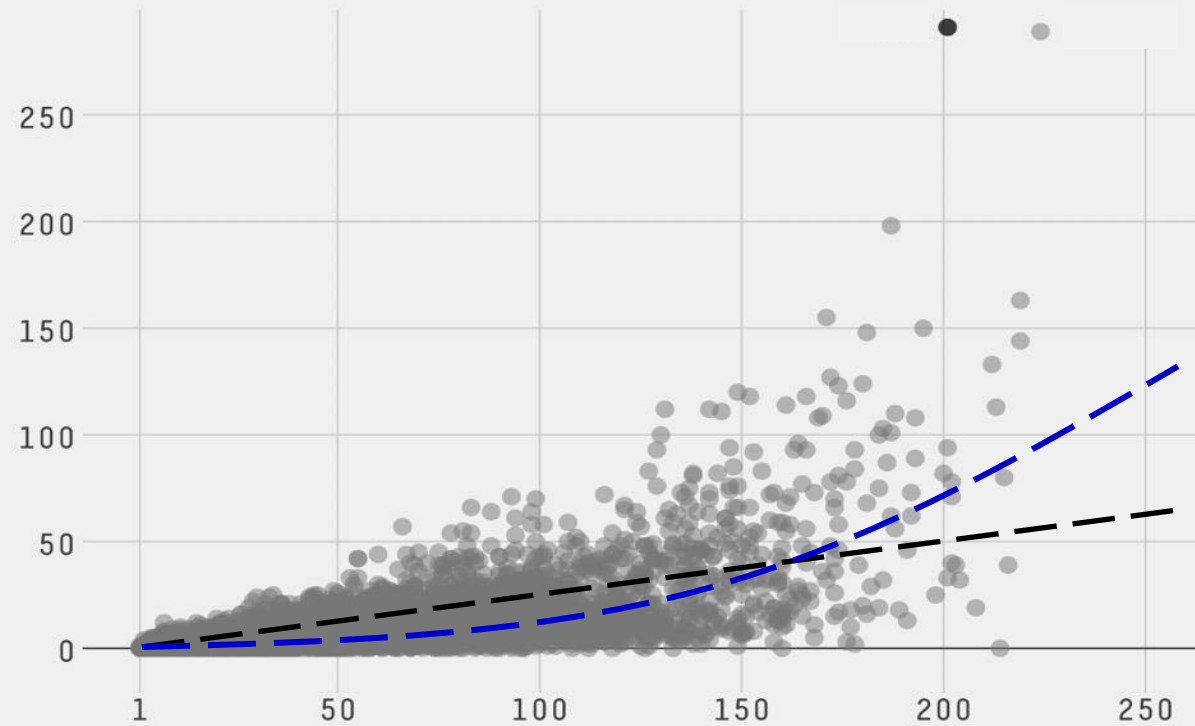


Pros	Cons
Short training time	Sensitive to noises and outliers, i.e. overfitting
Easy to interpret	Cannot handle complicated relationship, i.e. linear only
Easy to implement	

Regression: extension

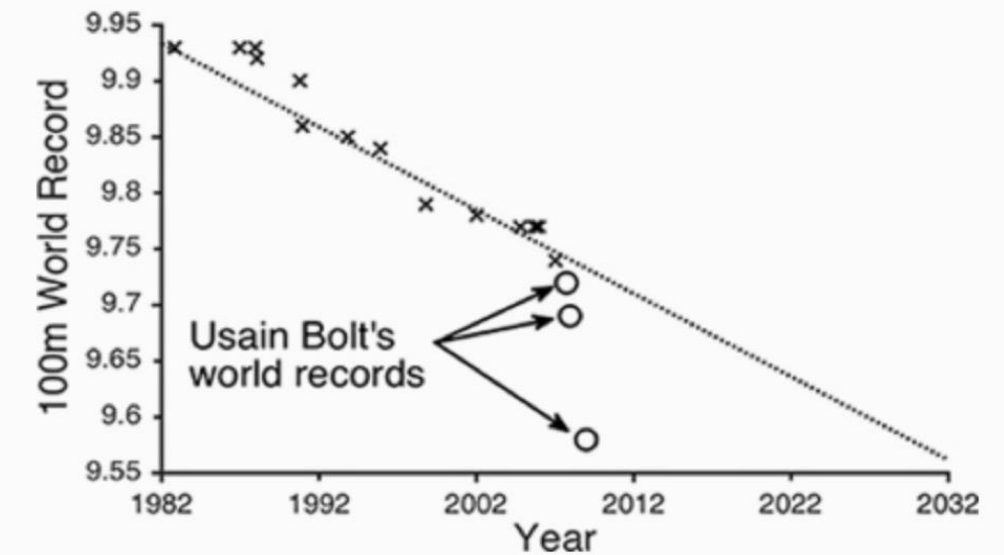
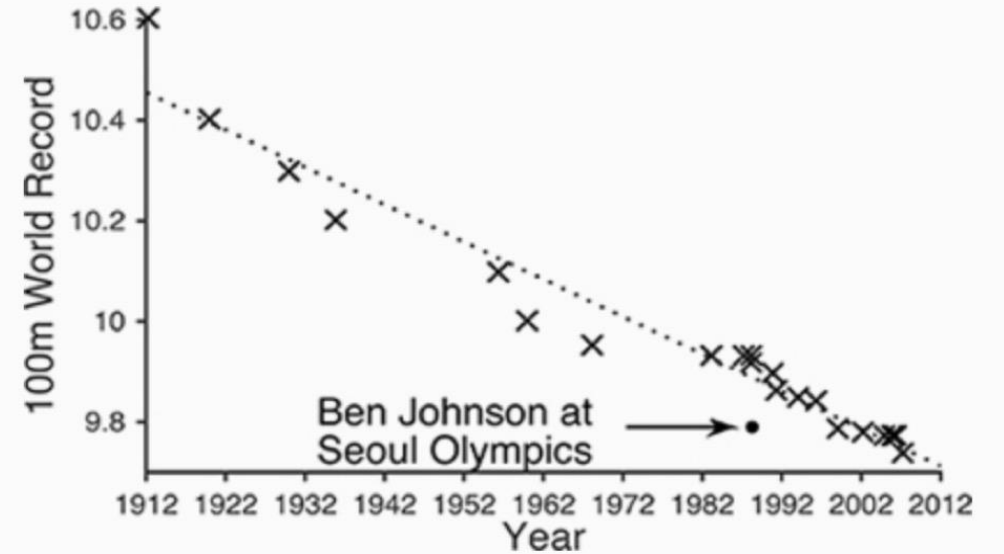
Overall Scoring Production

Total goals and assists vs. games played since 2010 World Cup



FIVETHIRTYEIGHT

SOURCE: ESPN/TRUMEDIA/OPTA



Regression: extension

Supervised

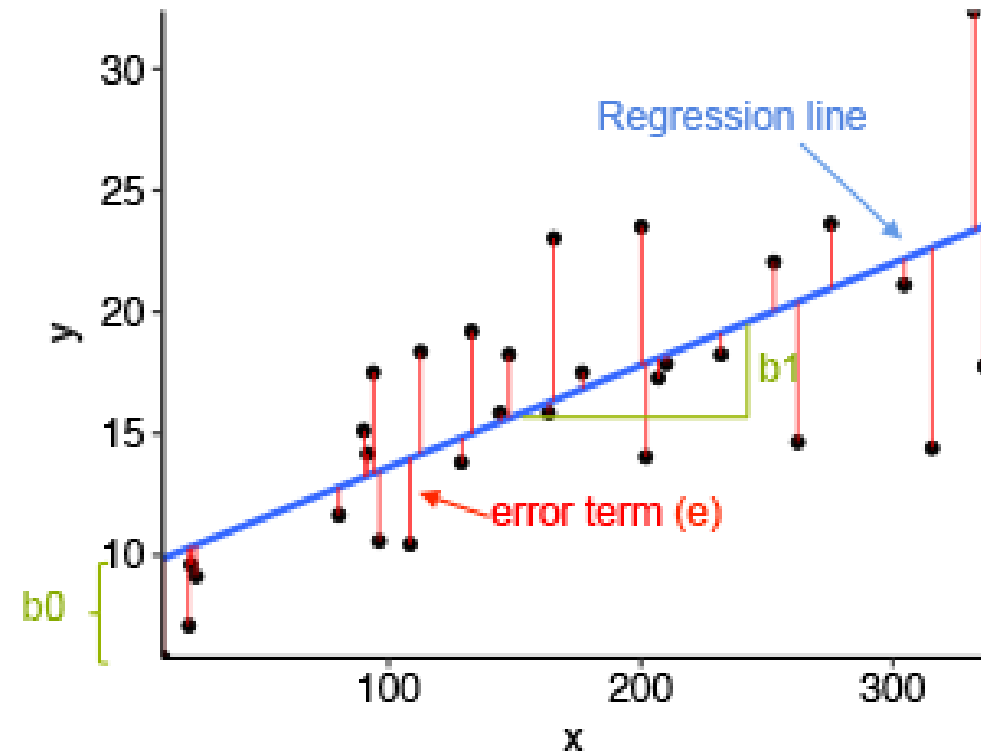
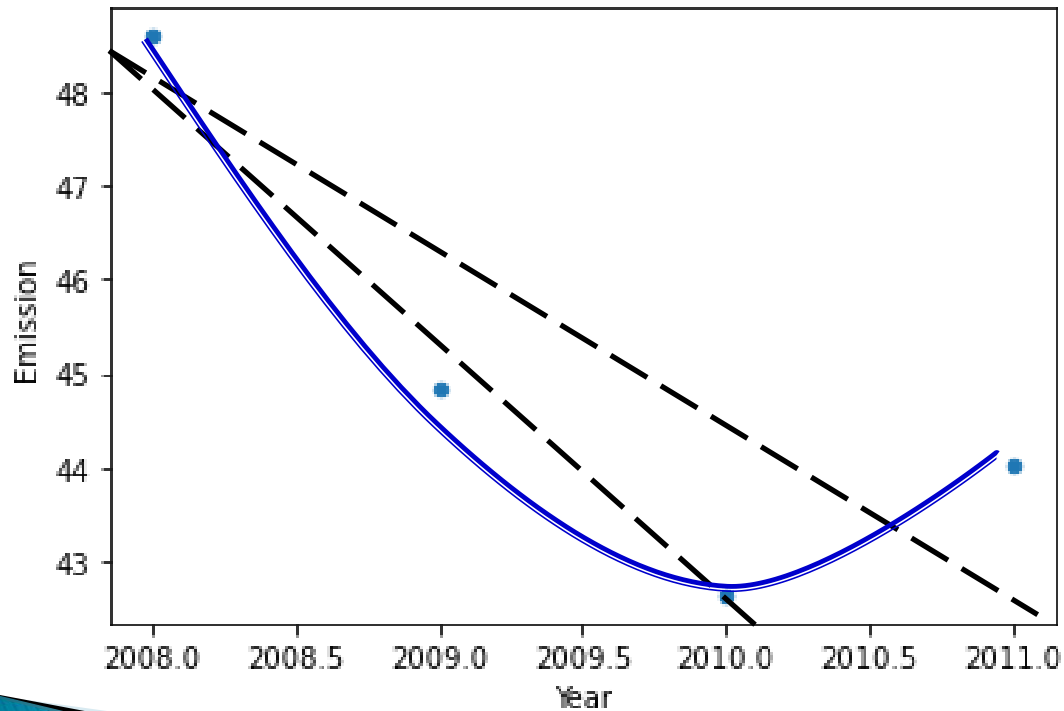
$y = F(x)$: relationship

D : labeled training set

Learn: $G(x)$: model to predict labels of new data

Goal: $[(F(x)-G(x))^2] \approx 0$

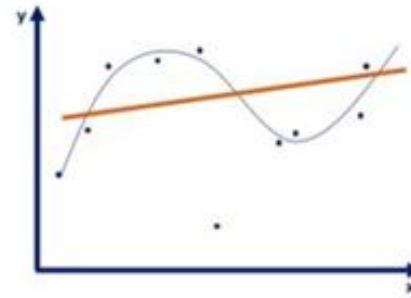
- Simple linear regression: not flexible
- Polynomial linear regression: $G(x) = b_0 + b_1 * x + b_2 * x^2$



Regression: extension

- ▶ Simple linear regression: not flexible
- ▶ Polynomial linear regression: $G(x)$
- ▶ Overfitting

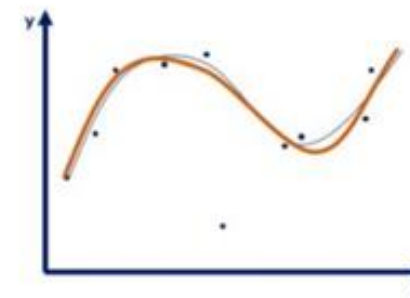
An **underfitted** model



Doesn't capture any logic

- High loss
- Low accuracy

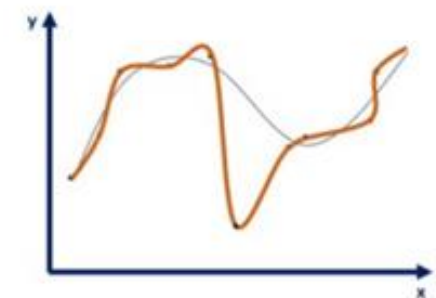
A **good** model



Captures the underlying logic of the dataset

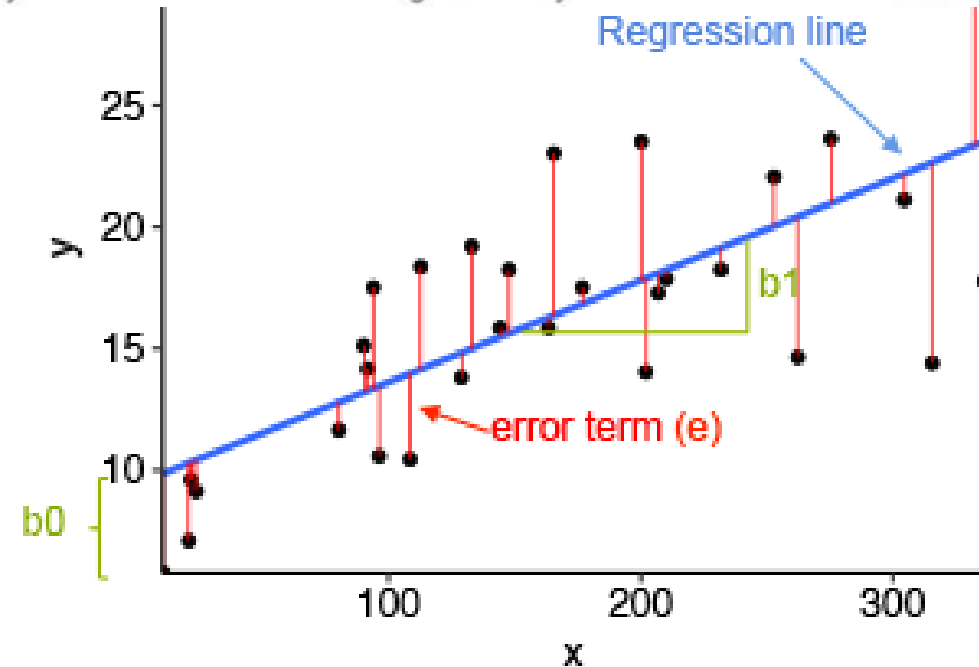
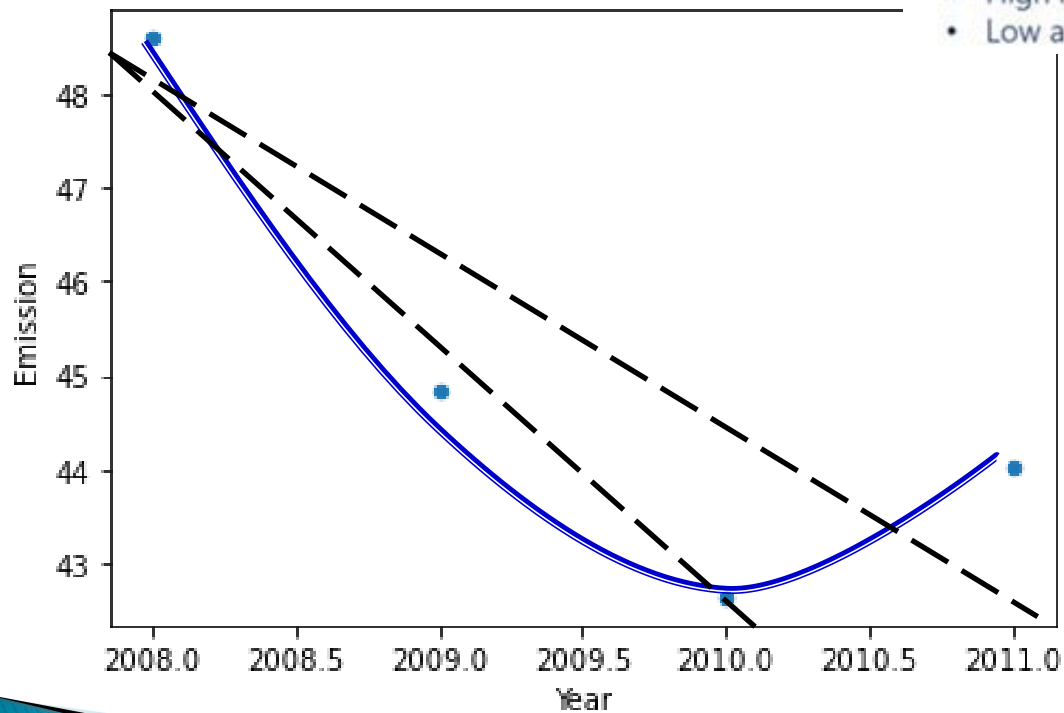
- Low loss
- High accuracy

An **overfitted** model



Captures all the noise, thus "missed the point"

- Low loss
- Low accuracy



Supervised Learning: Classification

Labels

- Categorical: **classification** (e.g. good, bad)
- Continuous: **regression** (e.g. probability)

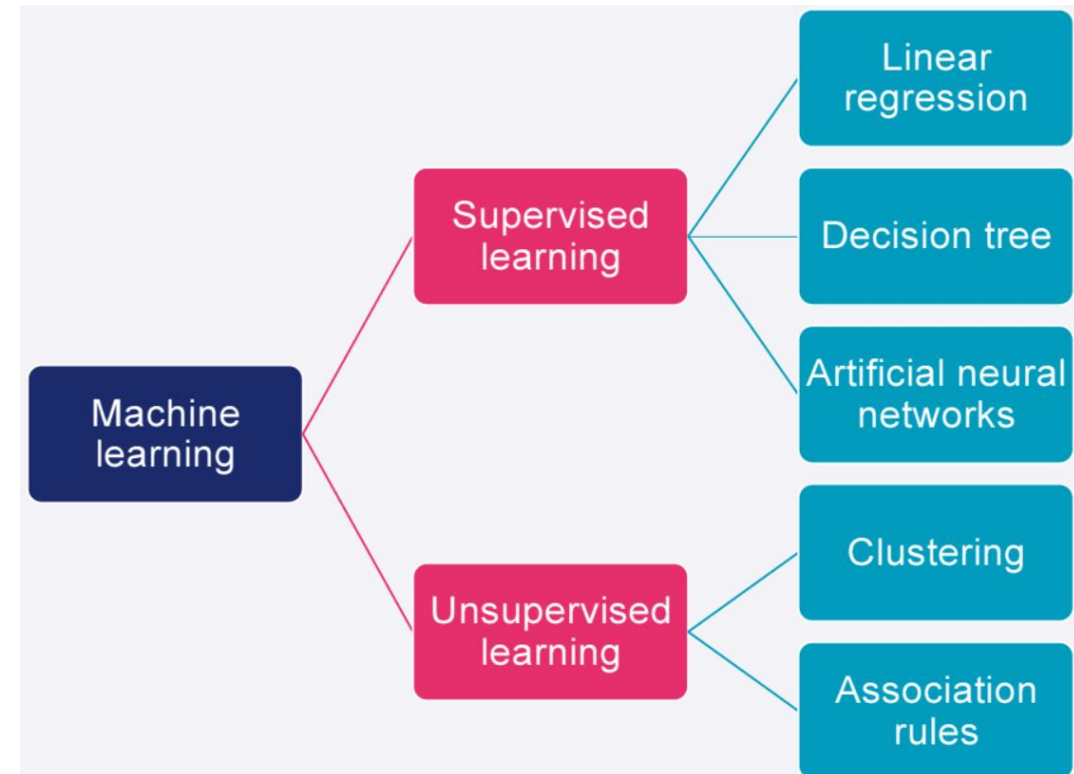
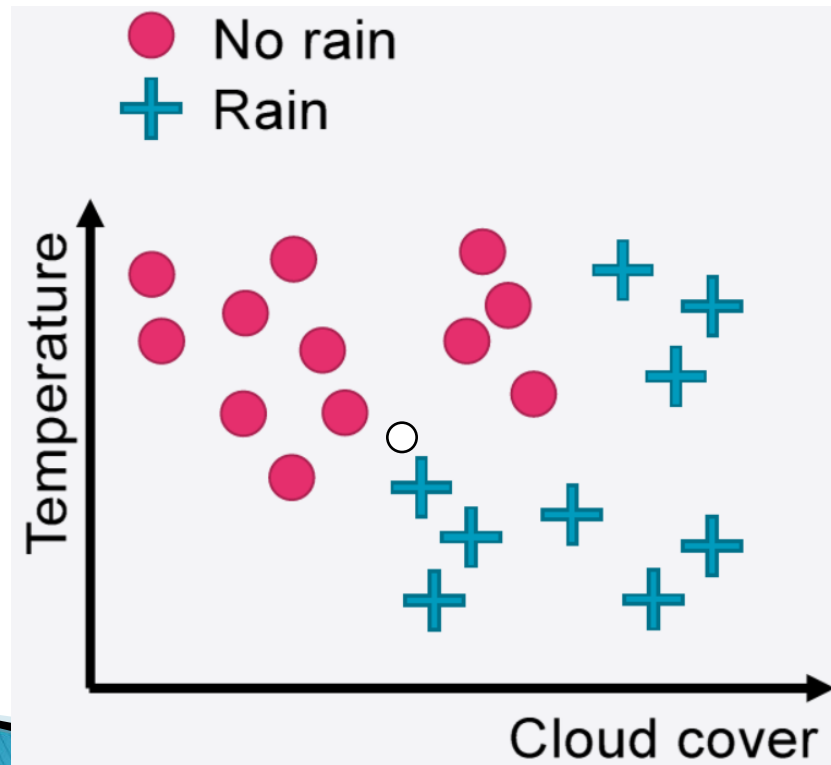
Supervised	Unsupervised
$y = F(x)$: relationship	$y = ?$: no function
D : labeled training set	D : unlabeled data
Learn : $G(x)$: model to predict labels of new data	Learn : ?
Goal : $[(F(x)-G(x))^2] \approx 0$	Goal : ?

1	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S
3	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
4	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S
5	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S
6	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05		S
7	6	0	3	Moran, Mr. James	male		0	0	330877	8.4583		Q
8	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625	E46	S
9	8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075		S
10	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333		S
11	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0	237736	30.0708		C
12	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7	G6	S
13	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	113783	26.55	C103	S
14	13	0	3	Saunders, Mr. William Henry	male	20	0	0	A/5. 2151	8.05		S
15	14	0	3	Andersson, Mr. Anders Johan	male	39	1	5	347082	31.275		S

1	PassengerId	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
751	750	3	Connaghton, Mr. Michael	male	31	0	0	335097	7.75		Q

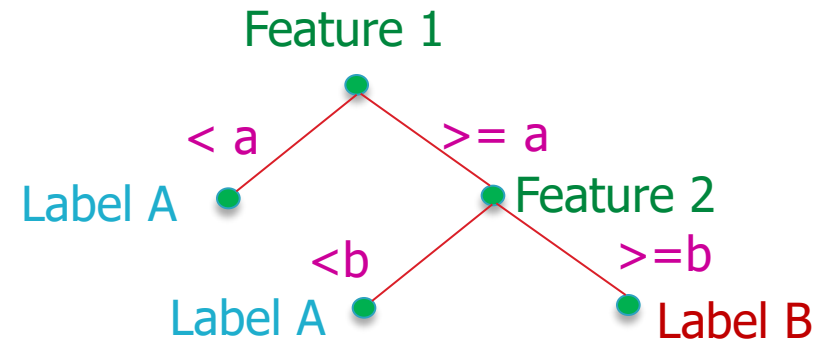
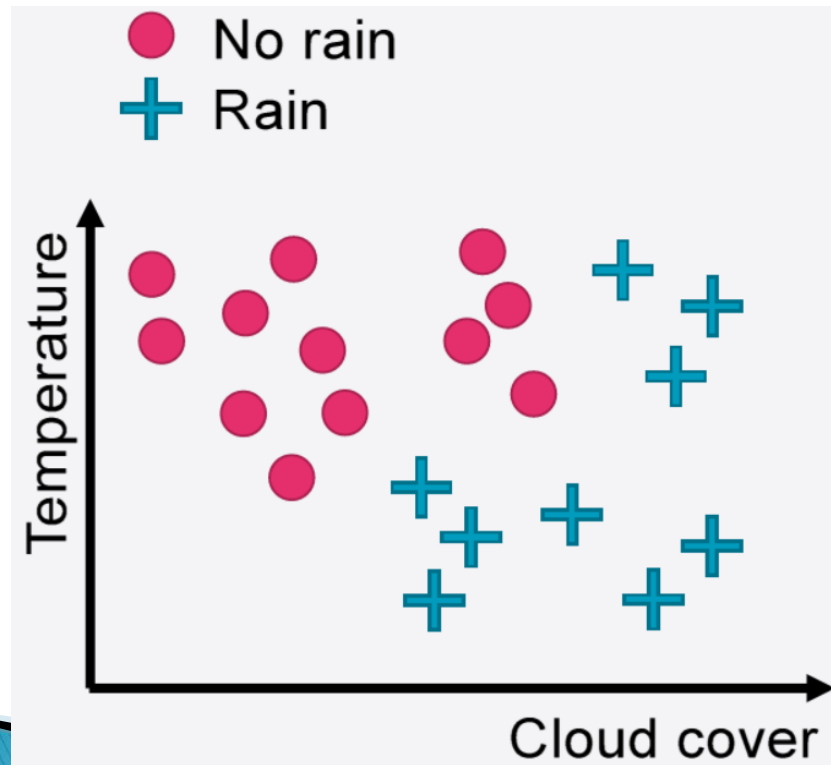
Classification

- Learn to predict to which set an instance belongs to based on **pre-labeled (classified)** instances



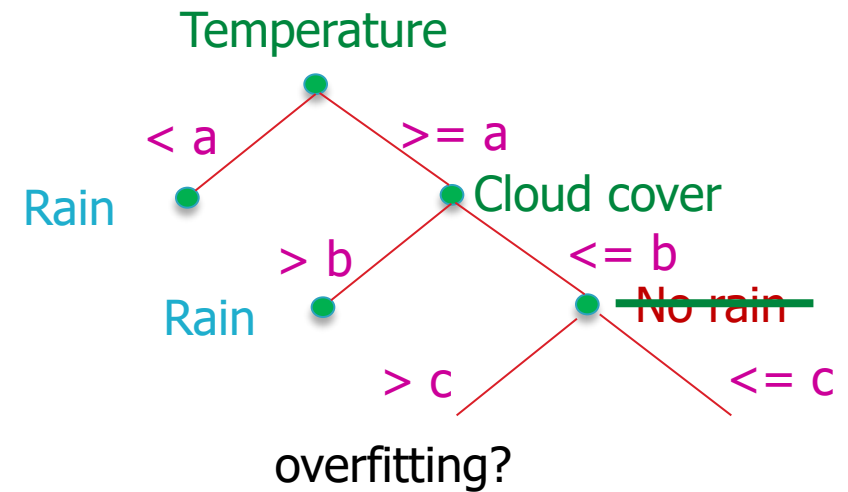
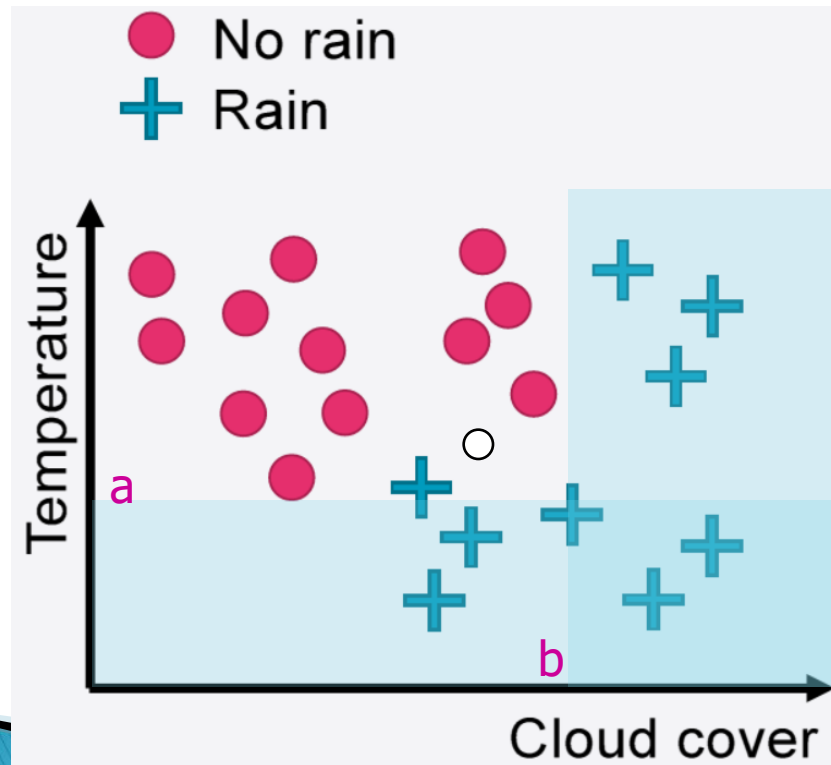
Decision Tree: Model

- ▶ **Internal nodes:** decision rules on **features** (decision variables, input)
- ▶ **Branches:** course of decision or action
- ▶ **Leaf nodes:** a predicted class **label** (output)



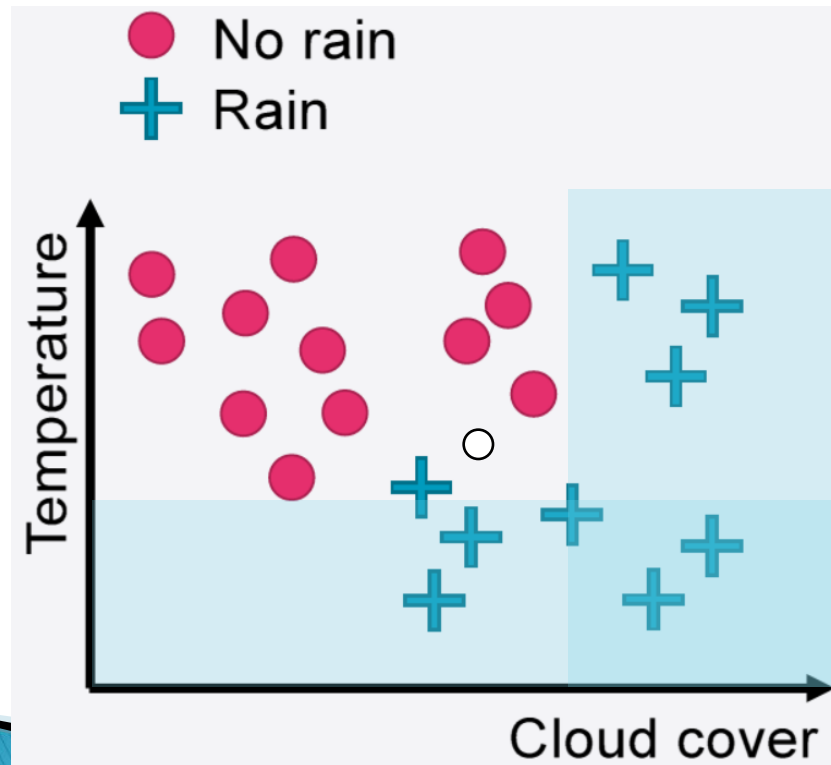
Decision Tree: Training & Testing

- ▶ Iteratively **partition** the decision space of chosen **features**
 - Which **features** should we split on?
 - Which **value** should we use for the split?



Decision Tree: Interpretation

- ▶ Which **features** should we split on?
- ▶ Which **value** should we use for the split?
- ▶ Overfitting? Practical?



Model 2 – Rule set 2:

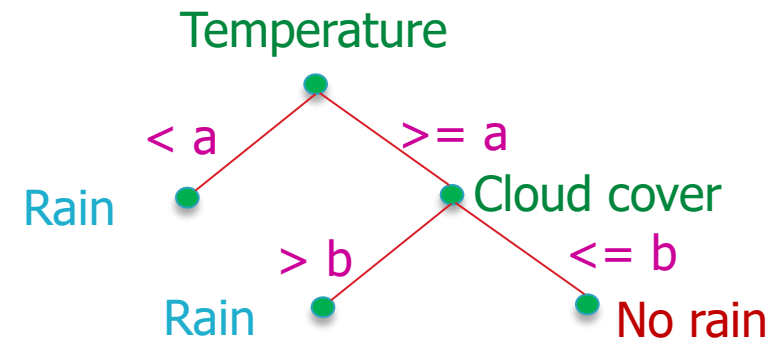
if **Temperature** < **a** then **Rain**

else if **Cloud cover** > **b** then **Rain**

else if **Cloud cover** < **c** **No rain**

else if **Temperature** > **d** **No rain**

else **Rain**



Model 1 – Rule set 1:

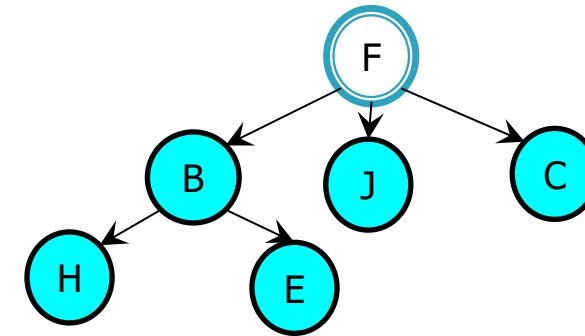
if **Temperature** < **a** then **Rain**

else if **Cloud cover** > **b** then **Rain**

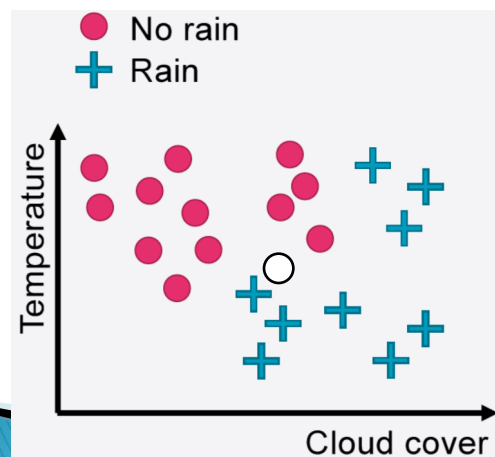
else **No rain**

Decision Tree: Model

- ▶ **Tree** – highly useful data structure
 - COMP1008 Part IV – path finding, game playing



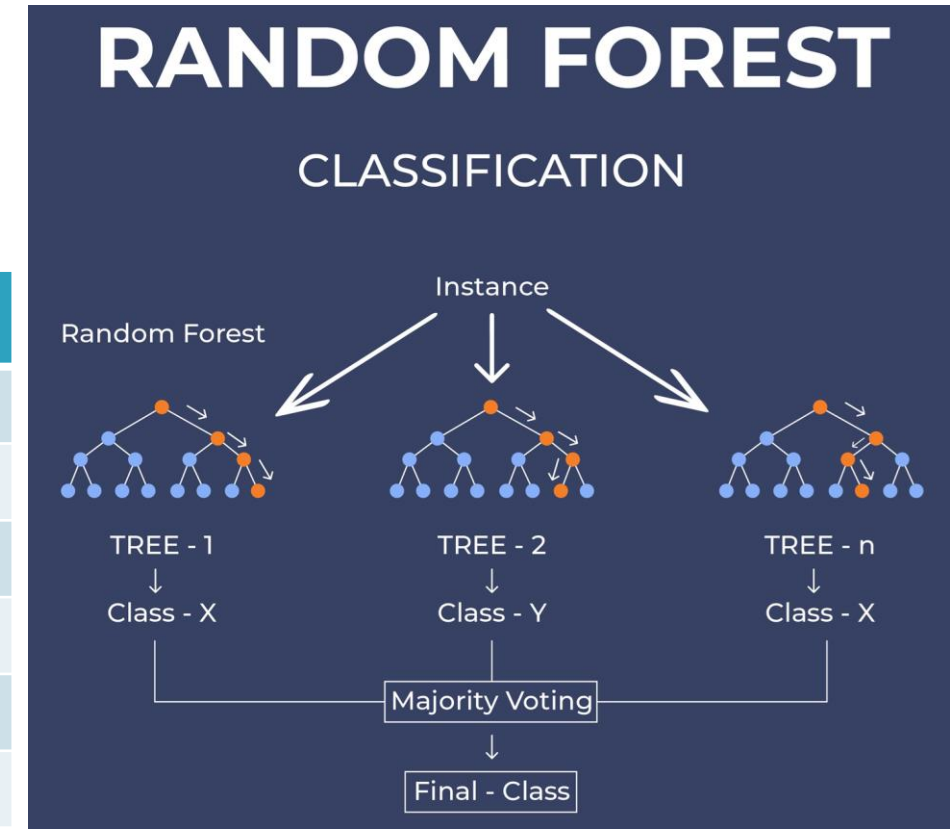
Pros	Cons
Reasonable training time	Only simple decision boundaries
Can handle large number of features	Problems with lots of missing data
Easy to implement	Cannot handle complicated relationship
Easy to interpret	Over-complex tree: overfitting



Decision Tree: Extension

- ▶ **Random Forest** – an ensemble method, constructing multiple trees
 - Each on random subsets of data
 - Each split considers a random subset of features
 - **Aggregates** predictions and importances from all trees

Decision Tree	Random Forest
Single tree	Multiple trees
Prone to overfitting	Reduces overfitting
Less accurate	Usually more accurate
Prone to noises and outliers	Handles well noises and outliers
Easy to interpret	Less interpretable
Reasonable training time	Longer training time



Decision Tree: The Titanic Example

Supervised	Unsupervised
$y = F(x)$: relationship	$y = ?$: no function
D : labeled training set	D : unlabeled data set
predict labels of new data	
Goal: $[(F(x)-G(x))^2] \approx 0$	Goal: ?

1	PassengerId	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
751	750	3	Connaghton, Mr. Michael	male	31	0	0	335097	7.75		Q

Labels

- Categorical: **classification** (e.g. good, bad)
- Continuous: **regression** (e.g. probability)

Features

1	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S
3	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
4	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S
5	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S
6	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05		S
7	6	0	3	Moran, Mr. James	male		0	0	330877	8.4583		Q
8	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625	E46	S
9	8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075		S
10	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333		S
11	10	1	2	Russel, Mrs. Nicholas (Adele Achen)	female	14	1	0	257730	30.0708		C
12	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7	G6	S
13	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	113783	26.55	C103	S
14	13	0	3	Saunderscock, Mr. William Henry	male	20	0	0	A/5. 2151	8.05		S
15	14		3	Andersson, Mr. Anders Johan	male	39	1	5	347082	31.275		S

Samples

Used to learn **model parameters**



How well does the model **generalize** on unseen test instances?

Decision Tree: Data Preparation

import useful libraries

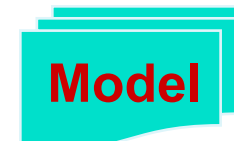
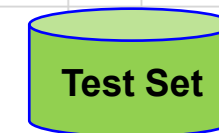
import pandas as pd

import matplotlib.pyplot as plt



Art by Simon Fisher www.simonfisherart.com

	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S
2	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
3	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S
4	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0		53.1	C123	S
5	5	0	3	Allen, Mr. William Henry	male	35	0	0		8.05		S
6	6	0	3	Moran, Mr. James	male		0	0		8.4583		Q
7	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0		51.8625	E46	S
8	8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1		21.075		S
9	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2		11.1333		S
10	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0		30.0708		C
11	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7	G6	S
12	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0		26.55	C103	S
13	13	0	3	Saunders, Mr. William Henry	male	20	0	0	A/5. 2151	8.05		S
14	14	0	3	Andersson, Mr. Anders Johan	male	39	1	5		31.275		S



Data Pre-processing



Machine Learning



Decision Tree: Data Preparation

```
df = pd.read_excel('data-Titanic.xlsx', index_col=0)
```

```
df_Y = df['Survived']
```

```
df_X = df[['Pclass', 'Gender', 'Age', 'SibSp', 'Fare']]
```

Labels

Features

1 PassengerId Survived Pclass Name Gender Age SibSp Parch Ticket Fare Cabin Embarked

2 1 0 3 Braund, Mr. Owen Harris male 22 1 0 A/5 21171 7.25 S

3 2 1 1 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female 38 1 0 PC 17599 71.2833 C85 C

4 3 1 3 Heikkinen, Miss. Laina female 26 0 0 STON/O2. 3101282 7.925 S

5 4 1 1 Futrelle, Mrs. Jacques Heath (Lily May Peel) female 35 1 0 113803 53.1 C123 S

6 5 0 3 Allen, Mr. William Henry male 35 0 0 373450 8.05 S

7 6 0 3 Moran, Mr. James male 0 0 330877 8.4583 Q

8 7 0 1 McCarthy, Mr. Timothy J male 54 0 0 17463 51.8625 E46 S

9 8 0 3 Palsson, Master. Gosta Leonard male 2 3 1 349909 21.075 S

10 9 1 3 Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg) female 27 0 2 347742 11.1333 S

11 10 1 2 Nasser, Mrs. Nicholas (Adele Achem) female 14 1 0 237736 30.0708 C

12 11 1 3 Sandstrom, Miss. Marguerite Rut female 4 1 1 PP 9549 16.7 G6 S

13 12 1 1 Bonnell, Miss. Elizabeth female 58 0 0 113783 26.55 C103 S

14 13 0 3 Saundercock, Mr. William Henry male 20 0 0 A/5. 2151 8.05 S

15 14 0 3 Andersson, Mr. Anders Johan male 39 1 5 347082 31.275 S

Samples

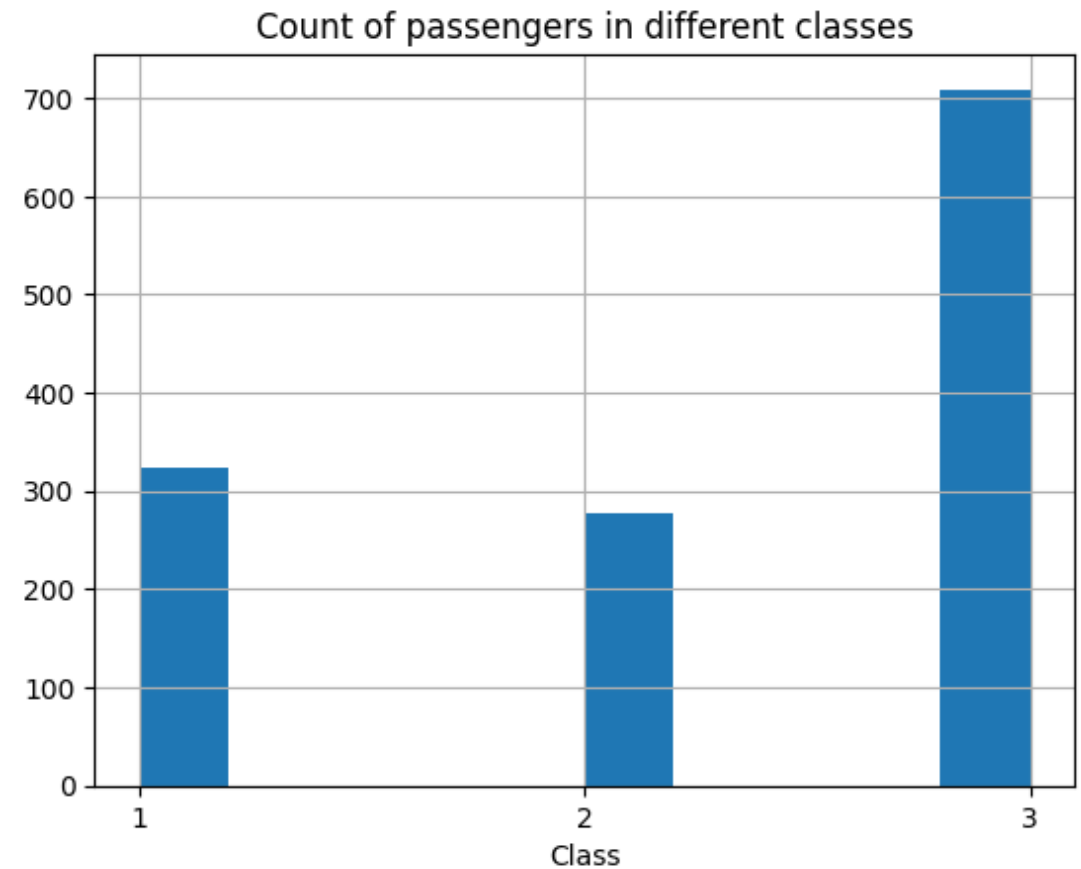
Decision Tree: Data Preparation

```
df_X['pclass'].hist()
```

```
plt.xlabel('Class')
```

```
plt.xticks([1,2,3])
```

```
df_Y[df_Y == 1].count() / df_Y.count()
```



Decision Tree: Data Preparation

data imputation: techniques which replace missing data

```
df_X.isnull().sum()
```

.mode(): returns a Series of the most frequent values

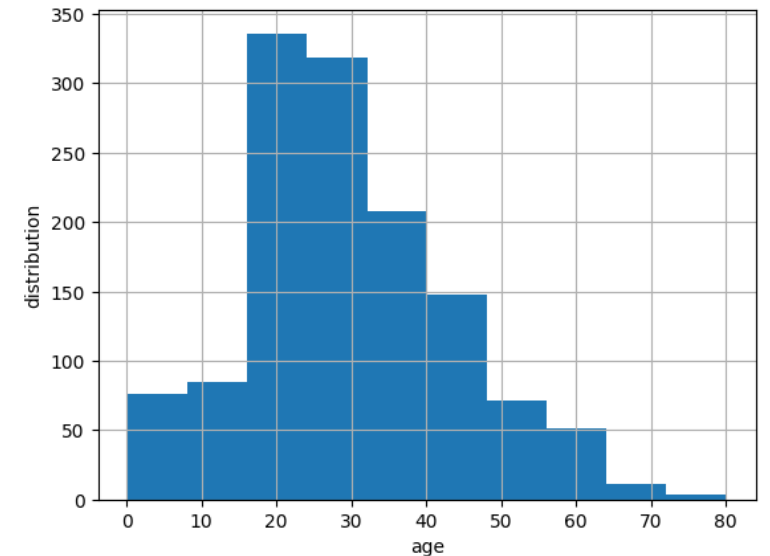
```
embarked_mode = df_X['Embarked'].mode()[0]
```

```
df_X['embarked'] = df_X['Embarked'].fillna(embarked_mode)
```

```
df_X['Fare'] = df_X['Fare'].fillna(df_X['Fare'].mean())
```

```
df_X['Age'] = ?
```

```
df_X['Embarked'] = pd.factorize(df_X['Embarked'])[0]
```



	pclass	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1.0	female	29.0000	0.0	0.0	24160	211.3375	B5	S
1	1.0	male	0.9167	1.0	2.0	113781	151.5500	C22 C26	S
2	1.0	female	2.0000	1.0	2.0	113781	151.5500	C22 C26	S
3	1.0	male	30.0000	1.0	2.0	113781	151.5500	C22 C26	S
4	1.0	female	25.0000	1.0	2.0	113781	151.5500	C22 C26	S
...	...	...	...	...	...	...	...	...	...
1304	3.0	female	14.5000	1.0	0.0	2665	14.4542	None	C
1305	3.0	female	NaN	1.0	0.0	2665	14.4542	None	C

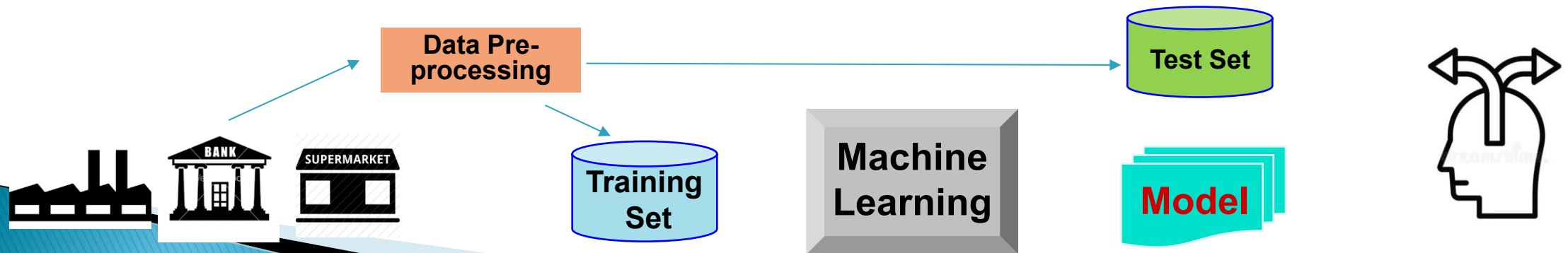
Decision Tree: Data Preparation

```
# import data partition methods
```

```
from sklearn.model_selection import train_test_split, cross_val_score
```

```
# split the data into subsets for training and testing
```

```
X_train, X_test, y_train, y_test = train_test_split(df_X, df_Y, test_size=0.3)
```



Decision Tree: Training

```
# import the machine learning package from the scikit learning library sklearn  
from sklearn.tree import DecisionTreeClassifier
```

y_train

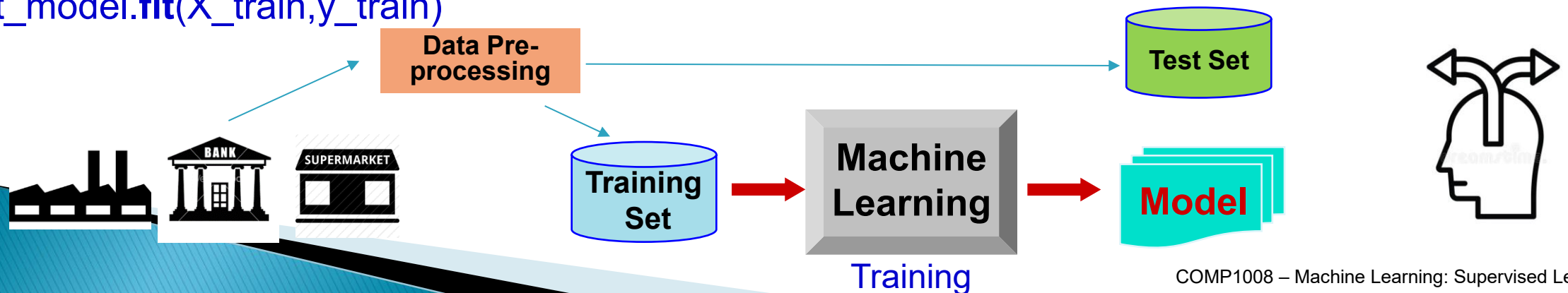
Survived	
0	0
1	1
1	2
1	3
0	4

X_train

pclass	sex	age	sibsp	parch	ticket	fare	cabin	embarked
1.0	female	29.0000	0.0	0.0	24160	211.3375	B5	S
1.0	male	0.9167	1.0	2.0	113781	151.5500	C22 C26	S
1.0	female	2.0000	1.0	2.0	113781	151.5500	C22 C26	S
1.0	male	30.0000	1.0	2.0	113781	151.5500	C22 C26	S
1.0	female	25.0000	1.0	2.0	113781	151.5500	C22 C26	S

t_model

```
t_model = DecisionTreeClassifier(max_depth = 4, random_state = 1) # randomness!  
t_model.fit(X_train, y_train)
```

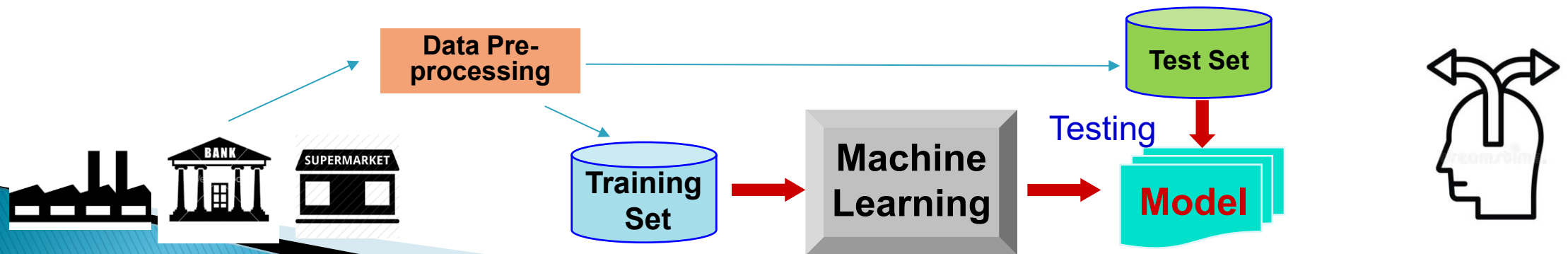
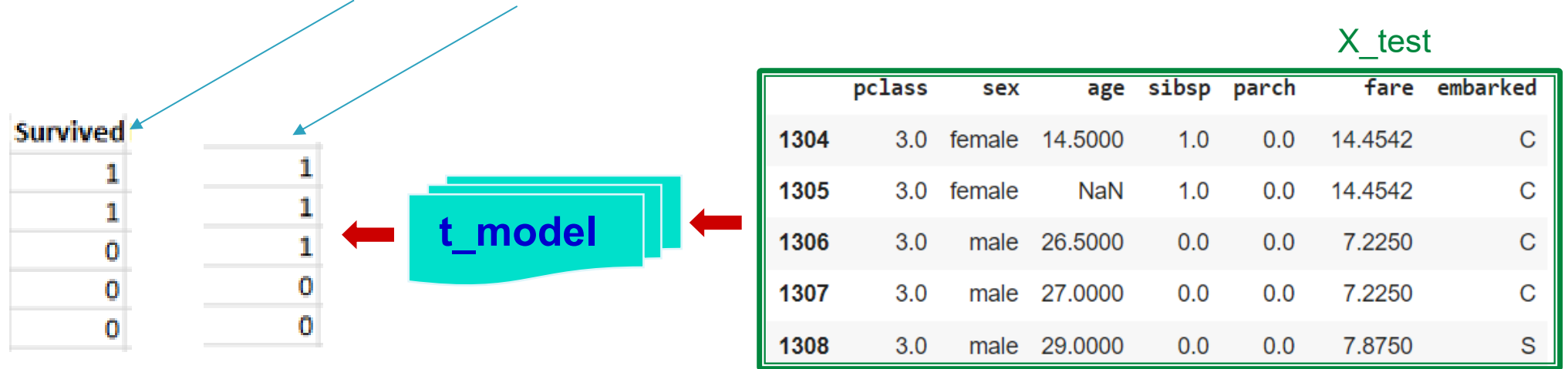


Decision Tree: Testing

```
y_predict = t_model.predict(X_test) # testing the trained decision tree model
```

```
from sklearn import metrics
```

```
print(metrics.accuracy_score(y_test, y_predict)) # evaluate the trained model on testing data
```

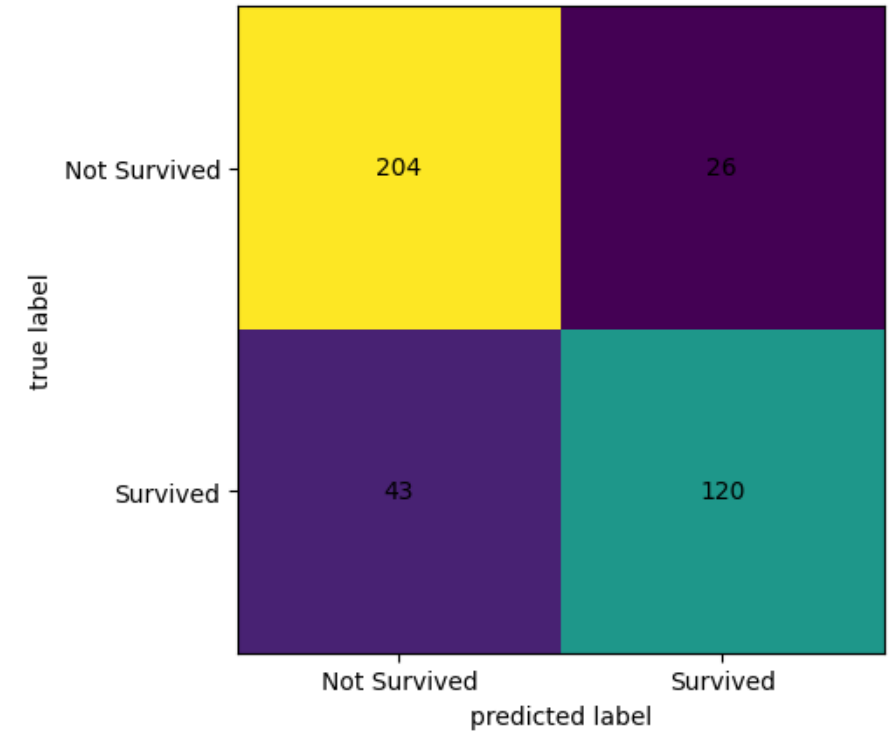


Decision Tree: Interpretation

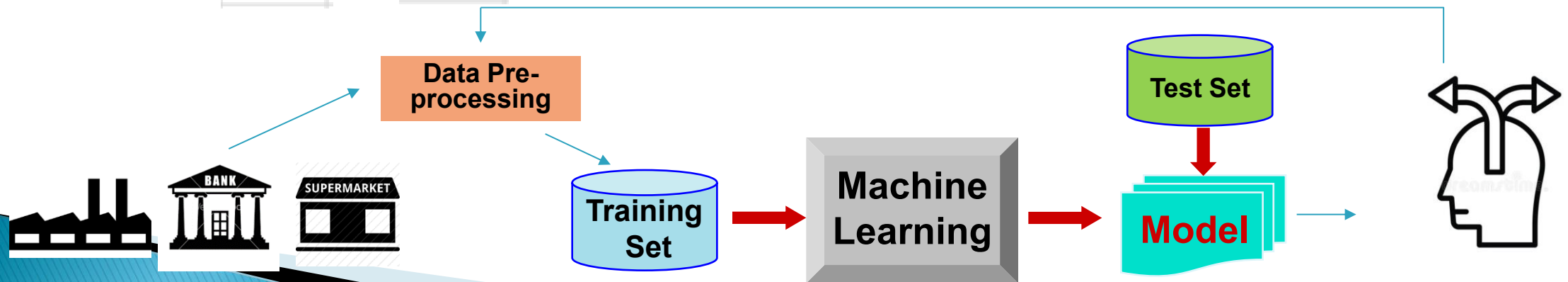
```
from sklearn.metrics import confusion_matrix  
cm_dt = confusion_matrix(y_test, y_predict)
```

decision parameter: depth

other decisions: features; missing data imputation



Survived	
1	1
1	1
0	1
0	0
0	0



Decision Tree: Interpretation

from sklearn import tree

plt.figure(figsize = (40,15))

tree.plot_tree(t_model, feature_names = df_X.columns, class_names=['Not survived', 'Survived'],
filled=True, fontsize=26)

plt.show()

t_model – Rule set 1:

if Sex $\leq$ 0.5

if pclass $\leq$ 2.5

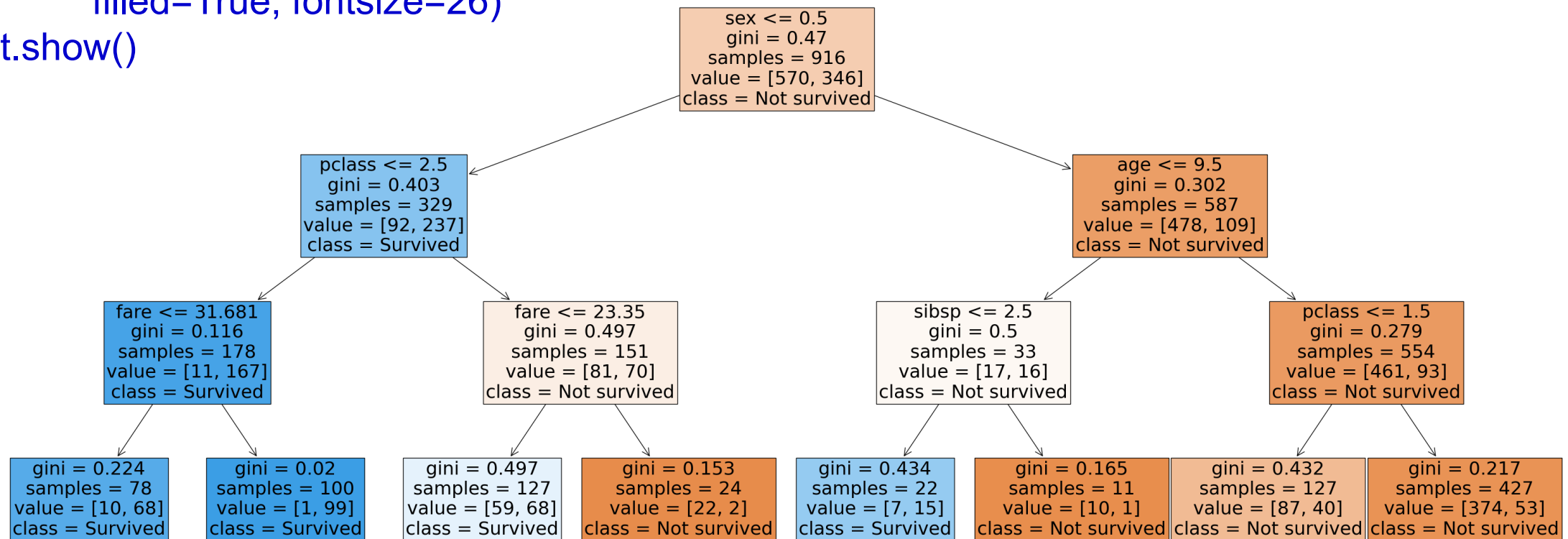
if fare $\leq$ 31.68 then Survived

else if fare $\leq$ 23.35 then Survived

else Not Survived

else if age $\leq$ 9.5

else ...



Machine Learning: Supervised Learning

- ▶ Learn/train models from **labelled** data, i.e. match output to inputs
 - Need high quality labelled data (very expensive!)
 - Known ground truth thus **metrics** for accurate evaluations
- Real-world applications
 - Healthcare: disease diagnosis, drug discovery
 - Retail: product recommendation, demand prediction
 - Transportation: traffic prediction, object recognition
 - Security: facial recognition, cyberattacks
 - NLP: translation, sentiment analysis
 - Finance: credit scoring for loan approvals
- Terminologies/process
 - Training/testing, concepts (overfitting, confusion matrix)
 - Decision tree, labels, features, classification/regression, etc.

